

ldmppr: Location Dependent Marked Point Processes in R

by Lane Drew and Andee Kaplan

Abstract In this article, we present *ldmppr*, an R package for estimating, evaluating, simulating from, and visualizing location-dependent marked spatial point processes. To date, it has commonly been assumed that the marks associated with a point process are independent of the locations. However, when dealing with many point processes, such as those arising in forestry applications, the independence assumption proves unreasonable. We introduce a practical framework for generating marked point processes with dependence between the marks and locations. We provide a brief discussion of the theory underpinning our modeling approach and outline the use of the package in a typical scenario involving real data. We highlight the functionality of the package for both generating from and assessing the goodness-of-fit of a given model, enabling users to generate realistic point patterns given a reference pattern or parameter values of interest.

1 Introduction

Point process models are a rich and complex class of models encompassing processes that occur over time, space, and potentially include additional information about the process in the form of marks. Marks, which are attributes associated with each point (i.e., size or type), add another layer of complexity, particularly when they depend on spatially or temporally varying covariates. The behavior of a point process is typically characterized by the relationship between points. For example, the simplest process is a homogeneous Poisson process, which assumes a constant intensity and no interaction between points and may be described as complete spatial randomness (CSR), where intensity describes the expected number of points per unit of area. More structurally involved processes may exhibit regularity (or inhibition), where points tend to repel each other, or clustering, where collections of points tend to occur in proximity to each other within the pattern. Capturing the dynamics of these processes can be challenging even without the inclusion of marks, and notably more so when this type of auxiliary information is present. Consequently, researchers often make the simplifying assumption that the marks associated with a point process are independent of the primary process itself. However, this assumption is often hard to justify from a scientific perspective, as in the case of trees within a forest where the sizes of the trees (i.e., canopy volumes) play a role in their distribution over space, and where the sizes themselves are likely to depend on location specific information such as elevation, soil moisture, or sunlight availability, which vary over space. Marked point processes have been used across a variety of fields including epidemiology, seismology, criminology, ecology, and the health sciences. Some examples of marked point processes include ambulance call locations with call severity and patient gender as marks (Bayisa et al., 2023), and locations of crime incidences with type of crime as a mark (Mohler et al., 2011).

In this paper, we present *ldmppr* (Drew and Kaplan, 2025), an R package that provides a suite of tools for working with location dependent marked point processes characterized by regularity in the point pattern. While a wealth of R packages exist for working with point processes, such as *spatstat* (Baddeley and Turner, 2005) and *ptProcess* (Harte, 2010), their focus is often broad in scope and they fail to incorporate dependence in the mark distribution with a high degree of flexibility. In contrast, *ldmppr* is designed specifically for working with spatial marked point processes with dependence between the marks and locations that demonstrate inhibitory behavior in the spatial pattern. The package is structured to deliver a straightforward and modular workflow that simplifies the process of model estimation, evaluation, simulation, and visualization given a reference dataset and a set of corresponding covariate surfaces in the form of raster images. In addition to the

included models, users can easily adapt the workflow to their specific needs by substituting appropriate components, while still taking advantage of the overall package structure.

While inhibitory marked point processes are commonly modeled using Gibbs processes, they are known to be computationally expensive and difficult to evaluate efficiently. Instead, we take a likelihood optimization approach that is computationally scalable, generally tractable, and enables a straightforward mechanism for simulating from and evaluating the goodness-of-fit of a model. To achieve this, we employ the mechanistic approach for equating a marked spatial point process with a spatio-temporal point process outlined by Møller et al. (2016), who demonstrated the utility of the self-correcting process introduced by Isham and Westcott (1979) for modeling marked point processes with regularity. We extend this framework to include location dependence in the mark distribution using a class of flexible non-linear models, improving the applicability of the original approach to a wider variety of biologically plausible and scientifically interesting processes.

The remainder of the paper is organized as follows. Section 2 provides an overview of the relevant point process theory and modeling framework underlying **ldmppr**. Section 3 describes the package structure and functionality. Section 4 showcases the standard workflow for the package on an example dataset, with an emphasis on forestry applications. Section 5 concludes with a discussion of the methodological contributions, strengths, limitations, and potential directions for future development of the package.

2 Mathematical background

We begin this section with a brief introduction of the theory of marked point processes to provide context for our modeling approach. We follow this with a discussion of our proposed approach for mapping a marked point process to a marked spatio-temporal process that preserves the dependence between marks and the process and the self-correcting model that we employ for point processes characterized by regularity.

2.1 Marked point processes

Marked point processes are an extension of point processes that include additional point specific information in the form of marks. Marks may represent continuous or discrete quantities (i.e., sizes or species of trees in a forest), and may be dependent or independent of the primary generating process. In a process with dependence, the distribution of the marks depends on the locations of the points, while independent marks occur independently of the points themselves. In practice, marked point processes may be modeled by a joint distribution for the points and their associated marks, or alternatively by the conditional distribution of the marks given the locations, often described by their first and second-order characteristics. The first order characteristic is the mark intensity function which describes the expected mark value per unit of area, while the second order characteristic, the mark correlation function, captures the dependence between marks at different times or locations (Schlather et al., 2004).

For purposes of illustration, we consider a marked spatial point process over a finite domain $\mathcal{S} \subset \mathbb{R}^2$ defined as $\Phi = \{(\mathbf{U}_i, \mathbf{M}_i) : i = 0, \dots, N\}$ with a two-dimensional nonnegative mark (though this may be extended to a higher dimensional mark space with additional real valued or discrete marks). We assume that the process is characterized by regularity such that points in the process tend to repel each other resulting in higher interpoint distances than would be observed in a pattern with CSR. We define $\mathbf{U}_i \in \mathcal{S}$ to be the spatial location of the i -th point and $\mathbf{M}_i = (M_{i1}, M_{i2}) \in \mathbb{R}^+ \times \mathbb{R}^+$ to be the mark vector associated with the i -th point, where $M_{i1} = M_1(\mathbf{U}_i)$ is taken to be a measure of size or age and $M_{i2} = M_2(\mathbf{U}_i)$ is a secondary mark that may be of additional scientific interest (i.e., height or dbh in forestry applications). We allow for the possibility that $M_{i1} = M_{i2}$, which may be of interest when utilizing our modeling approach to capture location dependence in the primary mark distribution. Additionally, $\mathbf{Z}(\mathbf{U}_i)$ represents a

vector of topographic (or location specific) covariates at location $\mathbf{U}_i$. We assume that the mark vector $\mathbf{M} = \{\mathbf{M}_i : i = 0, \dots, N\}$ is ordered according to the first mark such that the M_{i1} are increasingly ordered continuous random variables with $0 \leq M_{01} \leq \dots \leq M_{N1} \leq \tau < \infty$ and where τ is an unknown upper bound for the marks.

A common approach for modeling a process like Φ with regularity in the spatial pattern, would be to adopt a Gibbs process model as in [Møller and Waagepetersen \(2003\)](#). However, these models typically include an intractable normalizing constant that makes them difficult to estimate efficiently and often rely on computationally expensive MCMC algorithms. Alternatively, assuming that we have a point process with a structure like Φ , we can use a likelihood based approach that allows for straight forward estimation, model evaluation, and simulation by extending the mechanistic framework introduced by [Møller et al. \(2016\)](#). The method maps a spatial marked point process onto a marked spatio-temporal process conditional on the history of the process by incorporating location dependence in the mark distribution and allowing for higher dimensional marks, as described in the following section.

2.2 Spatio-temporal process mapping with location dependent marks

While marked point processes naturally capture the relationship between event locations and their associated attributes, directly modeling these dependencies can be challenging. By mapping a marked spatial point process onto a spatio-temporal process, we introduce a structured mechanism for estimation that preserves the dependence between marks and locations while leveraging established likelihood-based methods for estimation. Spatio-temporal point processes are a class of models that represent the occurrences of random events over time and space and may be extended to include additional marks that depend on space, time, or both domains simultaneously ([Rathbun and Cressie, 1994](#)). These processes are driven by a conditional intensity function

$$\lambda^*(t, \mathbf{y} \mid \mathcal{F}_t), \quad t > 0, \mathbf{y} \in \mathcal{S},$$

where $\mathcal{F}_t$ is the σ -algebra generated by the oldest point in the process $\mathbf{M}_N$, which we define as the anchor point for the process, and the points in the process $(T_i, \mathbf{Y}_i)$ with $T_i < t$ (a more thorough discussion may be found in [Daley and Vere-Jones \(1988\)](#)). The conditional intensity function describes the instantaneous expected rate of events at time t and at location $\mathbf{y}$, conditional upon all of the points in the process occurring before time t .

This process may be described as a mechanistic model ([Diggle, 2013](#)), and for a marked point process of the form of Φ , defined in Section 2.1, we can equate the process with an infinite marked spatio-temporal point process $\{(T_1, \mathbf{Y}_1, X_1), (T_2, \mathbf{Y}_2, X_2), \dots\}$, where $\mathcal{T} \subset \mathbb{R}$ is the temporal domain. To facilitate this transformation, we introduce a mapping from the original marked point process notation $\Phi = \{(\mathbf{U}_i, \mathbf{M}_i) : i = 0, \dots, N\}$ to a spatio-temporal representation $\{(T_i, \mathbf{Y}_i, X_i)\}$. First, we associate each point $(\mathbf{U}_i, \mathbf{M}_i)$ with a derived time $\tilde{T}_i \stackrel{\text{def}}{=} T(M_{i1}) \in \mathcal{T}$ via

$$\tilde{T}_i = 1 - \left[\frac{M_{i1} - \min_j(M_{j1})}{\max_j(M_{j1}) - \min_j(M_{j1})} \right]^\delta, \quad i = 0, \dots, N, \quad (1)$$

where δ controls the shape of the mapping function and $\delta = 1$ is a 1-to-1 linear mapping. This construction yields $\min_i \tilde{T}_i = 0$ and $\max_i \tilde{T}_i = 1$, so the observed times occur on the interval $[0, 1]$, which standardizes the times and ensures comparability across datasets for the temporal parameters while reducing sensitivity to the scale of the mark variable chosen for the temporal mapping.

We then relabel points according to their derived times, so that the spatio-temporal index corresponds to the ordered event times. Let π be a permutation of $\{0, \dots, N\}$ such that

$\tilde{T}_{\pi(1)} < \tilde{T}_{\pi(2)} < \dots < \tilde{T}_{\pi(N+1)}$. Define, for $i = 1, \dots, N+1$,

$$T_i \stackrel{\text{def}}{=} \tilde{T}_{\pi(i)}, \quad Y_i \stackrel{\text{def}}{=} U_{\pi(i)}, \quad X_i \stackrel{\text{def}}{=} M_{\pi(i),2}.$$

Thus, $Y_i \in \mathcal{S}$ is the spatial location of the i -th event in time, and X_i is the associated secondary mark, where X_i may depend on Y_i and $Z(Y_i)$. Under Equation (1) (for $\delta > 0$), larger primary marks correspond to smaller derived times, so ordering by T_i corresponds to processing points from largest to smallest M_{i1} , consistent with conditioning on the anchor point (U_N, M_N) . This representation allows us to generate a point pattern conditional on (U_N, M_N) , i.e., the point with the largest primary mark, where the behavior of the process after the final observed event time is ignored as we exploit the relationship between the observed finite processes.

We define the conditional intensity function of the our marked spatio-temporal process as follows

$$\lambda^*(t, \mathbf{y}, x \mid \mathcal{F}_t), \quad t > 0, \mathbf{y} \in \mathcal{S}, x \geq 0,$$

such that the intensity depends on the points arriving in the process before time t . The notation λ^* signifies the dependence on the history of the process, $\mathcal{F}_t$, which is suppressed in the notation going forward for concision. We note that under the assumption of conditional independence between the arrival times and spatial locations, the conditional intensity function may be decomposed as follows

$$\lambda^*(t, \mathbf{y}, x) = \lambda^*(t) h_t^*(\mathbf{y}, x), \quad t > 0, \mathbf{y} \in \mathcal{S}, x \geq 0,$$

where $\lambda^*(t)$ is the temporal intensity and $h_t^*(\mathbf{y}, x)$ is the likelihood for the marked spatial process. We note that $h_t^*(\mathbf{y}, x)$ may be decomposed further if we assume conditional independence between the locations and the secondary mark characteristic. This will extend the framework of Møller et al. (2016) to allow for spatio-temporal dependence in the secondary mark distribution and enable flexible estimation of the conditional mark distribution incorporating potentially complex location dependence.

We specify a general parametric model for the conditional intensity, i.e., $\lambda_\theta^*(t, \mathbf{y}, x)$, where θ is an unknown set of parameters and obtain the spatio-temporal log-likelihood conditional on $(U_N, M_N) = (u_n, m_n)$ such that

$$L(\theta) = \sum_{i=1}^n \log \lambda_\theta^*(t_i, \mathbf{y}_i, x_i) - \iiint_{(0, t_n) \times \mathcal{S} \times \mathbb{R}^+} \lambda_\theta^*(t, \mathbf{y}, x) dt d\mathbf{y} dx.$$

To facilitate model fitting and simulation, we introduce the temporal-integrated intensity function

$$\Lambda^*(t) = \int_0^t \lambda^*(s) ds, \quad t > 0, \quad (2)$$

such that $S_i = \Lambda^*(T_i)$, $i = 1, 2, \dots$ form a unit rate Poisson process on $(0, \infty)$. This relationship allows us to easily simulate from the process once the model is estimated, which is a key component in assessing the goodness-of-fit of the estimated process. Details for the simulation algorithm are provided in Section 2.5.

2.3 Model exhibiting regularity

The framework we have introduced in this section applies to a general marked point process. We now turn our attention to processes exhibiting regularity in their spatial pattern as described in Section 2.1. Point processes may be described as self-correcting when the intensity function decreases as the number of events increases, resulting in an inhibitory effect that counterbalances event clustering and enforces regularity in the spatial pattern over time. We define a modified version of the self-correcting model introduced by Isham

and Westcott (1979) with the following intensity function

$$\lambda^*(t, \mathbf{y}, x) = \lambda_{\theta_1}^*(t) h_{\theta_2, t}^*(\mathbf{y}) g_{\theta_3, t, \mathbf{y}, z(\mathbf{y})}^*(x) f_{\theta_4}^*(t, \mathbf{y}).$$

This model incorporates an arrival time process ($\lambda_{\theta_1}^*(t)$), spatial process ($h_{\theta_2, t}^*(\mathbf{y})$), conditional mark process ($g_{\theta_3, t, \mathbf{y}}^*(x)$), and spatio-temporal interaction ($f_{\theta_4}^*(t, \mathbf{y})$). We discuss the formulations for each component as follows.

Arrival time process

The arrival time process T is modeled with intensity function $\lambda_{\theta_1}^*(t)$, following Møller et al. (2016), given by

$$\lambda_{\theta_1}^*(t) = \exp(\alpha_1 + \beta_1 t - \gamma_1 N(t)),$$

where $\theta_1 = (\alpha_1, \beta_1, \gamma_1)$ such that $\alpha_1 \in \mathbb{R}$ is a baseline rate, $\beta_1 \in [0, \infty)$ is a log-linear function of t , and $\gamma_1 \in [0, \infty)$ is the scaling factor for $N(t)$ where $N(t) = \sum_{i=1}^N \mathbb{I}\{i \geq 0 : t_i < t\}$.

Spatial process

We model the spatial process with interaction function $\psi(r)$ such that

$$\psi_{\theta_2}(r) = \mathbb{I}[r \leq \alpha_2](r/\alpha_2)^{\beta_2} + \mathbb{I}[r > \alpha_2], \quad r \geq 0,$$

where r is the Euclidean distance between two points (i.e., $r_{ij} = \|\mathbf{y}_i - \mathbf{y}_j\|$). This formulation is analogous to the pairwise interaction function in Diggle and Gratton (1984). The density for the process is given by

$$h_{\theta_2, i}^*(\mathbf{y}_i) = \frac{1}{c_{\theta_2, i}^*} \prod_{j: j < i} \psi_{\theta_2}(r_{ij}), \quad (3)$$

where $c_{\theta_2, i}^*$ is the normalizing constant for the spatial density, which is defined as

$$c_{\theta_2, i}^* = \int_{\mathcal{S}} \prod_{j: j < i} \psi_{\theta_2}(r_{ij}) d\mathbf{y},$$

and $\theta_2 = (\alpha_2, \beta_2) \in [0, \infty)^2$. The spatial density $h_{\theta_2, i}^*$ defines an inhibitive circular region around each larger point $\mathbf{y}_j$, for all points with $t_j < t_i$, such that the strength of the interaction diminishes at a polynomial rate for interpoint distances less than α_2 and disappears for distances greater than α_2 .

Conditional mark process

We model the conditional mark process using a flexible non-linear model such that

$$g_{\theta_3}^*(x_i | t_i, \mathbf{y}_i, z(\mathbf{y}_i)) = G_{x_i}(t_i, \mathbf{y}_i, z(\mathbf{y}_i)),$$

where G is trained on the feature set $\{T_i, \mathbf{Y}_i, \mathbf{Z}(\mathbf{Y}_i)\}$, and $\mathbf{Z}(\mathbf{Y}_i)$ is the set of covariate values at the location $\mathbf{Y}_i$. We specify this component in generality, and note that any suitably flexible model may be chosen to model the conditional mark process (e.g., a random forest (Breiman, 2001) or gradient boosted tree model (Chen and Guestrin, 2016)). We assume that the mark process is conditionally independent given the times and locations and is parameterized by a non-overlapping set of parameters θ_3 when estimating the model. For example, in a forestry application, assuming conditional independence of marks (i.e., sizes) given spatial and temporal covariates implies that tree size at a given location and time is determined by local environmental factors and possible interpoint competition indices.

Spatio-temporal interaction

The final component of the model is the spatio-temporal interaction term, as introduced by Møller et al. (2016), which we define as

$$f_{\theta_4}^*(t_i, \mathbf{y}_i) = \exp \left(-\alpha_4 \sum_{j:j < i} \mathbb{I}[\|\mathbf{y}_j - \mathbf{y}_i\| \leq \beta_4, t_i - t_j \geq \gamma_4] \right),$$

where $\theta_4 = (\alpha_4, \beta_4, \gamma_4) \in [0, \infty)^3$ with $\beta_4 = \gamma_4 = 0$ if $\alpha_4 = 0$. This term connects the temporal and spatial process components of the full model and captures the dependence between these components. The form of the interaction term regulates the development of the process by limiting the occurrence of points that arrive in close proximity to each other in both space and time.

2.4 Parameter estimation

We now consider the procedure for estimating the parameters of the marked spatio-temporal process. We leverage the separability of the full log-likelihood under the assumption of conditional independence of the marks given the locations and arrival times and estimate the conditional mark process and the spatio-temporal process individually. This approach allows us to take advantage of highly flexible non-linear machine learning models to capture the conditional mark process, while maintaining the computational efficiency of maximum likelihood estimation for the spatio-temporal process. We note that both processes depend on the mapping from the original mark space to arrival times, which is controlled by the parameter δ in Equation (1). However, we do not necessarily require the mappings to be identical for both processes, as the conditional mark process may exhibit different behavior than the spatial process. We provide an overview of the estimation procedure for the full model in practice in Section 3.

For the self-correcting model described in Section 2.3, we estimate the parameters of the process by optimizing the spatio-temporal component of the log-likelihood, which may be expressed as

$$L(\theta_1, \theta_2, \theta_4) \propto \sum_{i=1}^n \left[\log \lambda_{\theta_1}^*(t_i) + \log h_{\theta_2,i}^*(\mathbf{y}_i) + \log f_{\theta_4}^*(t_i, \mathbf{y}_i) \right] - \iint_{(0,t_n) \times \mathcal{S}} \lambda_{\theta_1}^*(t) h_{\theta_2,t}^*(\mathbf{y}) f_{\theta_4}^*(t, \mathbf{y}) dt d\mathbf{y}.$$

The parameter sets $\theta_1, \theta_2, \theta_4$ can be estimated using any suitable optimization algorithm that allows for bound constraints on the parameter space.

For the conditional mark process, we estimate the parameter set θ_3 using a flexible model that allows for the incorporation of spatial and temporal covariates. The set of features used in training the model which correspond to location specific covariates $\mathbf{Z}(\mathbf{Y}_i)$ may be derived from raster images or other spatial data sources and may include interpoint competition metrics, as discussed in Section 3.3.

We combine the estimated parameter values for the conditional mark process and the spatio-temporal process to obtain the full set of estimated parameters for the marked spatio-temporal process. This approach is modular and allows for a wide variety of models to be substituted for the conditional mark process. In theory, this framework also holds for alternative spatio-temporal processes, such as those exhibiting clustering or other spatial patterns, though the specific form of the likelihood components would need to be adjusted accordingly.

2.5 Simulating from the spatio-temporal process

Utilizing the framework outlined in this section and estimates of the parameter sets $\theta_1, \theta_2, \theta_3, \theta_4$, we can easily generate realizations from the marked spatio-temporal point process on $[0, 1] \times \mathcal{S}$. Given $S_i = \Lambda^*(T_i)$, $i = 1, 2, \dots$ as defined in Equation (2), we have that

$$T_1 = \frac{1}{\beta_1} \log\{1 + \beta_1(\gamma_1 - \alpha_1)S_1\}$$

and for $i = 2, 3, \dots$

$$T_i = \frac{1}{\beta_1} \log \left[\exp(\beta_1 T_{i-1}) + \beta_1 \exp(\gamma_1 i - \alpha_1) S_i - \sum_{j=1}^{i-1} \{\gamma_1(i-j)\} \{\exp(\beta_1 T_j) - \exp(\beta_1 T_{j-1})\} \right]$$

such that we can generate a realization $t_1 < \dots < t_n$ under the self-correcting model on $[0, 1]$ as follows.

1. Generate a draw from a Poisson distribution with rate $\rho_{\hat{\theta}_1}^*(t) = \exp\{\hat{\alpha}_1 + \hat{\beta}_1 t_n\}$, where $t_n = 1$ to obtain the number of possible arrival times n^* .
2. For $i = 1, \dots, n^*$, generate a draw from the uniform distribution on $(0, t_n)$.
3. Order the uniform draws such that $u_1 < \dots < u_{n^*}$ to obtain the candidate arrival times $t_1^*, \dots, t_{n^*}^*$.
4. For $i = 1, \dots, n^*$, calculate the acceptance ratio

$$r_i^* = \frac{\lambda_{\hat{\theta}_1}^*(t_i^*)}{\exp\{\hat{\alpha}_1 + \hat{\beta}_1 t_n\}},$$

- then generate a draw u_i^* from the uniform distribution on $(0, 1)$ and accept t_i^* if $u_i^* < r_i^*$.
5. Given the set of accepted arrival times $\{t_i\}_{i=1}^n$, generate points $\mathbf{y}_i$ from the density $h_{\hat{\theta}_2, i}^*(\mathbf{y}_i)$ using rejection sampling with acceptance probability derived from the unnormalized component of the spatial interaction in Equation (3) and a uniform proposal distribution over $\mathcal{S}$.
6. Given the set of $\{t_i, \mathbf{y}_i\}_{i=1}^n$, thin the process such that each point is kept with probability $f_{\hat{\theta}_4}^*(t_i, \mathbf{y}_i)$ and the kept points are a realization of the spatio-temporal process.
7. For $i = 1, \dots, n$, given $(t_i, \mathbf{y}_i, \mathbf{z}(\mathbf{y}_i))$, generate x_i from the density $g_{\hat{\theta}_3}^*(x_i | t_i, \mathbf{y}_i, \mathbf{z}(\mathbf{y}_i))$.

This modeling approach allows us to employ interpretable and computationally tractable likelihood methods for estimating and simulating from the spatio-temporal process and provides a high degree of flexibility when selecting and training a model for the conditional mark distribution as discussed in the following section.

3 Package structure

In this section, we introduce the core functionality for the **ldmppr** package and detail the key functions associated with spatio-temporal process and mark model estimation, model goodness-of-fit evaluation, simulation, and visualization. We note that the package is currently designed for working with marked point processes that can be mapped onto spatio-temporal processes where the pattern is characterized by regularity and the mark distribution is dependent on location specific covariate information.

3.1 Workflow

We begin by describing the standard workflow that we envision for using the package. We decompose the task of working with marked point processes into a handful of straight-forward and manageable steps supported by an intuitive set of functions. The process is outlined as follows, where the output of each step can be passed as the input to successive stages where appropriate.

1. Estimate the parameters of a self-correcting point process given a reference dataset.
2. Train a mark model given the reference data set and topographic covariate surfaces in the form of rasters.
3. Check the fit of the estimated model using various non-parametric summaries for point processes and global envelope tests.
4. Simulate and visualize datasets from the fitted model.

We anticipate that users of the package will have a point process that they are interested in investigating, and we provide the tools to facilitate that exploration. We also note the modular structure of the package such that a user may provide their own estimated mark model in lieu of one of the currently available options in the package. In the remaining portions of this section, we detail the key functionality for each step given above.

3.2 Self-correcting model estimation

The first step in the model estimation procedure is to estimate the parameters of the self-correcting model detailed in Section 2.3 that captures the spatio-temporal process in the data. Given a reference dataset, we must define a mapping between our initial mark (i.e., size) and the arrival time in the process by specifying a value for δ . This allows the user to specify how the marks are mapped to arrival times using the transformation given in Equation (1). Once a mapping is chosen, the user should select an optimization strategy ("local", "global_local", or "multires_global_local") that determines how the optimization is performed. For the "local" strategy, the optimization is run using a single layer with a local algorithm. For the "global_local" strategy, the optimization is run using two layers that starts with a global optimization that is then updated in a local sweep. For the "multires_global_local" strategy, this extends the "global_local" approach to allow users to further refine the second stage local estimate at higher resolution. Once a strategy has been selected, users must specify both the integration grid and the optimization budget using `ldmppr_grids()` and `ldmppr_budgets()`, respectively. The grid controls the numerical resolution used to approximate the likelihood integral at each fitting level, while the budget controls how much optimization effort is allocated to each stage (global search, first-level local search, and refinement-level local search), including stopping criteria such as evaluation limits and convergence tolerances. In this context, "budget" refers to computational effort, not model complexity. With both components defined, parameter estimation for the self-correcting process proceeds via `estimate_process_parameters()`, where the arguments of the function are defined as follows.

Argument	Description
<code>data</code>	a data.frame or matrix. Must contain either columns (time, x, y) or (x, y, size). If a matrix is provided without time, it must have column names c("x", "y", "size").
<code>process</code>	type of process used (currently supports "self_correcting").
<code>grids</code>	a <code>ldmppr_grids</code> object specifying the integration grid schedule (single-level or multi-resolution). The integration bounds are taken from <code>grids\$upper_bounds</code> .
<code>budgets</code>	a <code>ldmppr_budgets</code> object controlling optimizer options for the global stage and local stages (first level vs refinement levels).

<code>parameter_inits</code>	(optional) numeric vector of length 8 giving initialization values for the model parameters. If <code>NULL</code> , defaults are derived from data and <code>grids\$upper_bounds</code> .
<code>delta</code>	(optional) numeric scalar or vector. Used only when data does not contain time (i.e., data has <code>(x,y,size)</code>). • If <code>length(delta) == 1</code>, fits the model once using <code>power_law_mapping(size, delta)</code>. • If <code>length(delta) > 1</code>, performs a delta-search by fitting the model for each candidate value and selecting the best objective. If <code>refine_best_delta = TRUE</code> and multiple grid levels are used, the best delta is refined on the remaining (finer) grid levels. If data already contains time, <code>delta</code> is ignored when <code>length(delta)==1</code> and is an error when <code>length(delta)>1</code>.
<code>parallel</code>	<code>TRUE</code> or <code>FALSE</code> specifying <code>furrr/future</code> to parallelize either: a) over candidate delta values (when <code>length(delta) > 1</code>), and/or b) over local multi-start initializations (when <code>starts\$local > 1</code>), and/or c) over global restarts (when <code>starts\$global > 1</code>).
<code>num_cores</code>	number of workers to use when <code>set_future_plan = TRUE</code> .
<code>set_future_plan</code>	<code>TRUE</code> or <code>FALSE</code> , temporarily sets <code>future::plan(multisession, workers = num_cores)</code> and restores the original plan on exit.
<code>strategy</code>	character string specifying the estimation strategy: • “local”: local optimization only (single-level or multi-level polish). • “global_local”: global optimization then local polish (single grid level). • “multires_global_local”: multi-resolution (coarsest uses global+local; refinements use local only).
<code>global_algorithm,</code> <code>local_algorithm</code>	NLopt algorithms to use for the global and local optimization stages, respectively.
<code>starts</code>	a list controlling restart and jitter behavior: • <code>global</code>: integer, number of global restarts at the first/coarsest level (default 1). • <code>local</code>: integer, number of local multi-starts per level (default 1). • <code>jitter_sd</code>: numeric SD for jittering (default 0.35). • <code>seed</code>: integer base seed (default 1).
<code>finite_bounds</code>	(optional) list with components <code>lb</code> and <code>ub</code> giving finite lower and upper bounds for all 8 parameters. If <code>NULL</code> , bounds are derived from <code>parameter_inits</code> . Global algorithms and select local algorithms in NLopt require finite bounds.
<code>refine_best_delta</code>	<code>TRUE</code> or <code>FALSE</code> . If <code>TRUE</code> and <code>length(delta) > 1</code> , performs refinement of the best delta across additional grid levels (if available).

<code>rescore_control</code>	controls candidate rescoreing and bound-handling behavior in multi-resolution fitting. Can be either: • a single logical value (toggle rescoreing on/off while keeping defaults), or • a named list with any of: <code>enabled</code>, <code>top</code>, <code>objective_tol</code>, <code>param_tol</code>, <code>avoid_bound_solutions</code>, <code>bound_eps</code>. Defaults are <code>list(enabled = TRUE, top = 5L, objective_tol = 1e-6, param_tol = 0.10, avoid_bound_solutions = TRUE, bound_eps = 1e-8)</code>.
<code>verbose</code>	TRUE or FALSE indicating whether to show progress of model estimation.

This function makes use of the R implementation of the NLOpt library ([Johnson, 2008](#)) and returns an `ldmppr_fit` object containing the estimated parameters and optimization details. The optimization procedure is designed to be flexible and allows for a variety of strategies and algorithm choices and combinations through the `nloptr` engine. The optimal choice of strategy and global/local algorithm may depend on the dataset, but the recommended default is a “`global_local`” strategy using the `NLOPT_GN_CRS2_LM` global algorithm paired with the `NLOPT_LN_BOBYQA` local algorithm. The `NLOPT_GN_CRS2_LM` algorithm is a global optimization algorithm that implements the “controlled random search” method, originally introduced by [Price \(1983\)](#), with the “local mutation” modification defined by [Kaelo and Ali \(2006\)](#). This algorithm is designed to be robust and efficient for global optimization problems, as it is less dependent on the initialization values and has been found to perform well in our testing. The recommended local optimization algorithm `NLOPT_LN_BOBYQA` is a derivative-free optimization algorithm that is designed for bound-constrained optimization problems through iteratively building a quadratic approximation of the objective function ([Powell, 2009](#)). Another local optimization option is `NLOPT_LN_SBPLX`, which is an implementation of the “Subplex” algorithm introduced by [Rowan \(1990\)](#) that incorporates explicit bound constraints. This algorithm is a more efficient and robust implementation of the original Nelder-Mead algorithm ([Nelder and Mead, 1965](#)). If the user wishes to test a set of different δ values for the mapping function, the `delta` argument may be specified as a vector and the `refine_best_delta` argument enables refinement of the optimal δ from the initial stage in additional layers.

In practice, we recommend a staged setup for `estimate_process_parameters()`. First, set `delta = 1` and obtain a baseline fit with a coarse grid and moderate optimization budgets. If diagnostics suggest poor fit, then increase optimization effort by adding additional starts and moving to the “`multires_global_local`” strategy with stricter tolerances. If users wish to tune `delta`, in the absence of prior knowledge of the relationship, we recommend searching a small neighborhood around 1 (rather than a wide range), selecting the best objective, and then refining only that candidate with finer grids. This strategy typically improves robustness while keeping computational cost manageable and allows users to refine the fit as needed based on the available diagnostics.

In addition to the optimization strategy, the user may also specify the number of global and local multi-starts and the amount of jittering to add to the initial parameter values for each start within the `starts` argument. Jittering multiple starts can help to improve the optimization results, and reduce dependence on the initial values. This will increase the computational time for estimation, though this can be offset by parallelization over the multi-starts using the `parallel` and `num_cores` arguments. The `estimate_process_parameters()` function is constructed to derive reasonable initial values and bound constraints based on the provided reference dataset, and we encourage users to use these defaults unless they have substantial knowledge about the reasonable values and parameter bounds for their specific application. We also note that the success of the optimization procedure may depend on the number of function evaluations and tolerance settings for the various algorithms,

which can be controlled through the `budgets` argument. We recommend starting with a moderate number of function evaluations (e.g., 1000 for the global stage and 500 for the local stage) and adjusting as needed based on the results and computational resources available. The choice of integration grid granularity, specified through `grids`, for the spatial and temporal components should reflect the scale of the data and the expected range of interactions. See Section 4 for an example of the effect of the choice of strategy, grids, and budgets on the model fit.

3.3 Mark model training

The second step in the model estimation procedure is to train the conditional mark model. We use the reference data with the mapped arrival times derived in the self-correcting model estimation step, or an alternate mapping, in concert with a set of covariate surfaces in the form of raster images. The raster images may be pre-processed using the `scale_rasters()` function, or may be provided in their raw form. To train the model, we use the `train_mark_model()` function, where the arguments of the function are provided as follows.

Argument	Description
<code>data</code>	a <code>data.frame</code> or a <code>ldmppr_fit</code> object.
<code>raster_list</code>	a list of raster objects.
<code>scaled_rasters</code>	TRUE or FALSE indicating whether the rasters have been scaled.
<code>model_type</code>	the machine learning model type (“ <code>xgboost</code> ” or “ <code>random_forest</code> ”).
<code>xy_bounds</code>	a vector of domain bounds (2 for x, 2 for y). If <code>data</code> is an <code>ldmppr_fit</code> and <code>xy_bounds</code> is NULL, defaults to <code>c(0, b_x, 0, b_y)</code> derived from fit.
<code>delta</code>	(optional) numeric scalar used only when <code>data</code> contains (x,y,size) but not time. If <code>data</code> is an <code>ldmppr_fit</code> and time is missing, the function will infer the <code>delta</code> value from the fit.
<code>save_model</code>	TRUE or FALSE indicating whether to save the generated model.
<code>save_path</code>	the path for saving the generated model.
<code>parallel</code>	TRUE or FALSE indicating whether to use parallelization in model training.
<code>num_cores</code>	number of cores to use in parallel model training (if <code>parallel</code> is TRUE).
<code>include_comp_inds</code>	TRUE or FALSE indicating whether to generate and use competition indices as covariates.
<code>competition_radius</code>	distance for competition radius if <code>include_comp_inds</code> is TRUE.
<code>edge_correction</code>	type of edge correction to apply (“ <code>none</code> ”, “ <code>toroidal</code> ”, or “ <code>truncation</code> ”).
<code>selection_metric</code>	metric to use for identifying the optimal model (“ <code>rmse</code> ”, “ <code>mae</code> ”, or “ <code>rsq</code> ”).
<code>cv_folds</code>	number of cross-validation folds to use in model training. If <code>cv_folds <= 1</code> , tuning is skipped and the model is fit once with default hyperparameters.
<code>tuning_grid_size</code>	size of the tuning grid for hyperparameter tuning.
<code>seed</code>	integer seed for reproducible resampling/tuning/model fitting.
<code>verbose</code>	TRUE or FALSE indicating whether to show progress of model training.

The `train_mark_model()` function allows users to select between a random forest or gradient boosted tree model, using the **ranger** (Wright and Ziegler, 2017) and **xgboost** (Chen et al., 2026) engines respectively, where these models are effective at capturing potentially complex non-linear relationships. Users choose a model selection criterion, either root mean squared error (RMSE) or mean absolute error (MAE), and may employ cross validation and hyperparameter tuning using a grid design that optimizes the maximum entropy of the hyperparameter space. As a general rule, we recommend that users start with a moderate number of cross-validation folds (e.g., 5) and a tuning grid size of 100, and adjust as needed based on the results and computational resources available, where parallelization can be used to speed up the tuning process. The `train_mark_model()` function is designed to be flexible and allows users to specify a variety of options for model training, including the ability to save the trained model for future use.

The function also allows users to incorporate a collection of interpoint competition metrics at a specified neighborhood size to capture additional trends that are not accounted for by the topographic covariates. The metrics include nearest neighbor distance, number of neighbors, average neighbor distance, nearest neighbor arrival time, sum of neighbor arrival times, and the ratio of nearest neighbor distance and arrival time. For an in depth discussion of competition indices and their construction, see Pommerening and Sánchez Meador (2018) and Contreras et al. (2011). Finally, users may select an edge correction mechanism when training the model to account for the possibility that the reference dataset provided is a subset of a larger dataset and that unobserved points are impacting the observed mark values (i.e., sizes).

3.4 Goodness-of-fit checks for the fitted model

Once the self-correcting model parameters are estimated and the mark model is trained, we turn our attention to assessing how well the fitted models capture the dynamics observed in the reference dataset. To accomplish this, we use the `check_model_fit()` function, which provides global envelope tests, using the **GET** package (Myllymäki et al., 2017), for a collection of standard non-parametric marked point process summary functions. We include the L , F , G , J , E , and V functions as evaluation metrics, together with a combined global envelope test that provides a single Monte Carlo p -value across the full set of summaries.

The L -function is a variance-stabilized transformation of Ripley's K -function and is commonly interpreted as a second-order measure of spatial interaction: relative to CSR, $L(r)$ above (below) its theoretical reference suggests clustering (inhibition) at spatial scale r (Ripley, 1976, Diggle (2013)). The F and G functions summarize complementary nearest-neighbor behavior: $F(r)$ (the empty-space function) describes distances from arbitrary locations in $\mathcal{S}$ to the nearest event, whereas $G(r)$ (the nearest-neighbor distribution function) describes distances from events to their nearest neighboring event (Diggle, 2013). The J -function combines these summaries via $J(r) = \frac{1-G(r)}{1-F(r)}$; values $J(r) > 1$ are typically associated with inhibition/regularity and values $J(r) < 1$ with clustering (van Lieshout, 2006). To assess the mark component, we use the mark summary functions E and V (implemented as `E`mark and `V`mark in **spatstat**), which describe how the conditional mean and conditional variability of marks change with interpoint distance, thereby providing diagnostics for mark–location dependence beyond the unmarked spatial structure (Schlather et al., 2004, Baddeley et al. (2015)). Finally, global envelope testing provides a principled way to control multiplicity across distances (and across multiple functions in the combined test) when comparing the observed summaries to simulations from the fitted model (Myllymäki et al., 2017).

The arguments of the `check_model_fit()` function are defined as follows.

Argument	Description
----------	-------------

reference_data	(optional) a marked ppp object for the reference dataset. If NULL, the reference pattern is derived from process_fit when process_fit is an ldmppr_fit and contains data_original (preferred) or data with columns (x, y, size).
t_min	minimum value for time.
t_max	maximum value for time.
process	type of process used (currently supports “self_correcting”).
process_fit	either an ldmppr_fit object (from estimate_process_parameters) or a numeric vector of length 8 giving the process parameters.
anchor_point	(optional) vector of (x, y) coordinates of the point to condition on. If NULL, inferred from the reference data (largest mark if available) or from the ldmppr_fit.
raster_list	(optional) a list of raster objects used for predicting marks. Required when mark_mode = “mark_model” unless rasters are stored in mark_model.
scaled_rasters	TRUE or FALSE indicating whether rasters are already scaled. Ignored when mark_mode = “time_to_size”.
mark_model	(optional) a mark model object used when mark_mode = “mark_model”. May be an ldmppr_mark_model, model_fit, or workflow.
xy_bounds	(optional) vector of bounds as c(a_x, b_x, a_y, b_y). If NULL, inferred from reference_data’s window when reference_data is provided; otherwise inferred from the ldmppr_fit with lower bounds assumed to be 0.
include_comp_inds	TRUE or FALSE indicating whether to compute competition indices.
competition_radius	distance for competition radius if include_comp_inds = TRUE.
thinning	TRUE or FALSE indicating whether to use the thinned simulated values.
edge_correction	type of edge correction to apply (“none” or “toroidal”).
n_sim	number of simulated datasets to generate.
save_sims	TRUE or FALSE indicating whether to save and return the simulated metrics.
verbose	TRUE or FALSE indicating whether to show progress of model checking. When TRUE, progress is reported via progressr (if available) and is compatible with parallel execution.
seed	integer seed for reproducibility.
parallel	TRUE or FALSE. If TRUE, simulations are run in parallel via furrr/future.
num_cores	number of workers to use when parallel = TRUE. Defaults to one fewer than the number of detected cores.
set_future_plan	TRUE or FALSE. If TRUE and parallel = TRUE, set a temporary future plan internally and restore the previous plan on exit.
mark_mode	(optional) mark generation mode. “mark_model” uses predict() on a mark model, while “time_to_size” maps simulated times back to sizes via delta. If NULL, inferred as “mark_model” when mark_model is provided, otherwise “time_to_size”.
fg_correction	correction used for F/G/J summaries (“km” or “rs”).
max_attempts	maximum number of simulation attempts when rejection occurs due to non-finite summaries.

The `check_model_fit()` function allows users to simulate a collection of datasets using the estimated parameters from the self-correcting process and trained mark model, which can be passed directly as `ldmppr_fit` and `ldmppr_mark_model` objects through the `process_fit` and `mark_model` arguments respectively. The non-parametric summary functions are calculated for each dataset and the global envelopes are obtained across the whole collection of simulated datasets for each metric, as well as a combined test across all metrics. This allows the user to gauge whether the realizations from the estimated location dependent marked point process accurately reflect the dynamics observed in the reference dataset across a variety of different metrics. Each individual metric includes a p -value range indicating the compatibility of the reference dataset with the simulated datasets, and the combined test provides a single p -value across the entire set of metrics allowing users to quickly gauge the overall fit of the model. Contrary to many testing procedures, a non-significant result (i.e., a large p -value) indicates that the reference dataset is not incompatible with the fitted model, whereas a significant result (i.e., a small p -value) indicates that the reference dataset is incompatible with the fitted model. We recommend starting with a moderate number of simulations (e.g., 200 or 500) to balance accuracy of the p -value estimates with computational time, and adjusting as needed based on the results and resources available. Parallelization can be employed to speed up the simulation and testing process when a larger number of simulations is desired. For additional discussion of the summary functions included in `check_model_fit()` see [Møller and Waagepetersen \(2003\)](#) and [Schlather et al. \(2004\)](#).

3.5 Simulation and visualization

When a user is satisfied with the estimated location dependent marked point process obtained in the model estimation, training, and goodness of fit checking steps, they can proceed with simulating realizations from the process and visualizing the results. The `simulate_mpp()` function provides an efficient way to generate a realization from the process, where the arguments for the function are as follows.

Argument	Description
<code>process</code>	type of process used (currently supports “self_correcting”).
<code>process_fit</code>	either (1) an <code>ldmppr_fit</code> object returned by <code>estimate_process_parameters</code> , or (2) a numeric vector of length 8 giving self-correcting parameters $(\alpha_1, \beta_1, \gamma_1, \alpha_2, \beta_2, \alpha_3, \beta_3, \gamma_3)$.
<code>t_min</code>	minimum value for time.
<code>t_max</code>	maximum value for time.
<code>anchor_point</code>	(optional) vector of (x, y) coordinates of the point to condition on. If NULL, inferred from the reference data (largest mark if available) or from <code>process_fit\$data_original</code> (largest size).
<code>raster_list</code>	(optional) list of raster objects used for mark prediction. Required when <code>mark_mode</code> = “mark_model” unless rasters are stored in <code>mark_model</code> .
<code>scaled_rasters</code>	TRUE or FALSE indicating whether rasters are already scaled. Ignored when <code>mark_mode</code> = “time_to_size”.
<code>mark_model</code>	(optional) mark model object used when <code>mark_mode</code> = “mark_model”. May be an <code>ldmppr_mark_model</code> , <code>model_fit</code> , or <code>workflow</code> .
<code>xy_bounds</code>	(optional) vector of bounds as <code>c(a_x, b_x, a_y, b_y)</code> . If NULL, bounds are inferred from <code>process_fit</code> when available.
<code>include_comp_inds</code>	TRUE or FALSE indicating whether to compute competition indices.

competition_radius	distance for competition radius if include_comp_inds = TRUE.
thinning	TRUE or FALSE indicating whether to use the thinned simulated values.
edge_correction	type of edge correction to apply (“none” or “toroidal”).
seed	integer seed for reproducibility.
mark_mode	(optional) mark generation mode. “mark_model” uses predict() on a mark model, while “time_to_size” maps simulated times back to sizes via delta. If NULL, inferred as “mark_model” when mark_model is provided, otherwise “time_to_size”.
size_range	(optional) numeric vector c(smin, smax) used for mark_mode = “time_to_size”. If NULL, inferred from process_fit when possible.
delta	(optional) positive scalar used for mark_mode = “time_to_size”. If NULL, inferred from process_fit when possible.

The function returns a `ldmppr_sim` object containing the relevant generating information, which can be provided to the function directly or inferred by passing a `ldmppr_fit` and `ldmppr_mark_model` through the `process_fit` and `mark_model` arguments respectively, and realized simulation in both a `ppp` format and as a `data.frame`. With a realization of the process in hand, the user can easily visualize the marked point process object using the default `plot()` method, which has the following arguments.

Argument	Description
<code>x</code>	an <code>ldmppr_sim</code> object
<code>pattern_type</code>	type of pattern to plot “simulated” (default).

In the following section, we walk through the entire workflow described in Section 3.1 with an example forestry dataset.

4 Application

Equipped with an understanding of the package workflow and primary functionality, we now provide an example of using `ldmppr` to analyze a forest stand dataset comprised of canopy volumes (in cubic meters) and locations for conifer species in the Southern Rocky Mountains, obtained from [Drew et al. \(2024\)](#), that is included in the package. We incorporate four topographic covariates, in the form of raster surfaces, that have been previously found to be related to the processes of tree growth and that capture key environmental conditions like energy and water availability ([Drew et al., 2025](#)). The covariates included in this analysis are Southness Aspect, Topographic Wetness Index, Elevation, and Slope which are derived from a LiDAR based digital elevation model (DEM).

Following the steps outlined in Section 3, we begin by estimating the self-correcting model using $\delta = 1$ as the size to time mapping parameter. We load the data and specify the mapping as follows.

```
data("medium_example_data")
parameter_estimation_data <- medium_example_data

delta <- 1
```

Next, we define the grid values for the spatial and temporal components of the process and upper bounds using the `ldmppr_grids()` function, and the budgets for the optimization

procedure using the `ldmppr_budgets()` function. To get a sense of where to start with the grid values, we recommend plotting the data and considering the spatial and temporal scales of the observed dynamics. For example, in this dataset, the spatial domain is approximately 50m by 50m, and the mapped arrival times range from 0 to 1. We want to ensure that the grid values capture the relevant scales of the observed dynamics, while balancing computational cost. The nearest neighbor distance distribution in our dataset can be used to identify a reasonable starting point for the spatial grid values.

```
summary(spatstat.geom::nndist(parameter_estimation_data[, -3]))
```

```
#>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>  0.9415  2.3064  2.7598  2.9095  3.4448  6.4035
```

Given this distribution, we start with a single grid level, specified as a $10 \times 10 \times 10$ grid for the spatial and temporal components, respectively. We opt to employ the “global_local” strategy for optimization, which includes a global optimization stage followed by a local polish. We start with a moderate number of function evaluations and higher tolerance for both the global and local stages to facilitate a low cost initial estimation.

```
upper_bounds <- c(1, 50, 50)
```

```
grids <- ldmppr_grids(
  upper_bounds = upper_bounds,
  levels = list(c(10, 10, 10))
)
```

```
budgets <- ldmppr_budgets(
  global_options = list(
    maxeval = 500,
    ftol_rel = 1e-3,
    xtol_rel = 1e-3
  ),
  local_budget_first_level = list(
    maxeval = 1000,
    ftol_rel = 1e-4,
    xtol_rel = 1e-4
  )
)
```

```
starts <- list(global = 1, local = 1, jitter_sd = 0.1, seed = 90210)
```

Using the `estimate_process_parameters()` function, we then estimate the parameters of the self-correcting process and obtain the optimal parameter estimates. We use the `NLOPT_GN_CRS2_LM` global and `NLOPT_LN_BOBYQA` local algorithms (described in Section 3.2) with the internally derived initial values and parameter bounds based on the reference data for the optimization. Details regarding the derivation of the initial values and bounds can be found in Appendix A of the Supplementary Materials.

```
estimated_sc <- estimate_process_parameters(
  data = parameter_estimation_data,
  process = "self_correcting",
  grids = grids,
  budgets = budgets,
  delta = 1,
  parallel = FALSE,
  strategy = "global_local",
```

```

    global_algorithm = "NLOPT_GN_CRS2_LM",
    local_algorithm = "NLOPT_LN_BOBYQA",
    starts = starts,
    verbose = FALSE
  )

optimal_parameters <- coef(estimated_sc)
summary(estimated_sc)

#> Summary: ldmppr Fit
#>   process:      self_correcting
#>   engine:       nloptr
#>   strategy:     global_local
#>   starts:       global=1, local=1, jitter_sd=0.35, seed=90210
#>   status:       3
#>   outcome:      NLOPT_FTOL_REACHED: Optimization stopped because ftol_rel or
#>                  ftol_abs (above) was reached.
#>   objective:    432.2978
#>   selected_delta: 1
#>   elapsed_sec:   0.038
#>   coefficients:
#> [1] 4.516536e+00 2.495385e-01 8.525285e-08 1.129760e-07 1.105553e-07
#> [6] 1.095338e-07 1.067978e-07 1.205356e-01

```

We obtain the estimated parameter set ($\alpha_1 = 4.5165$, $\beta_1 = 0.2495$, $\gamma_1 = 0$, $\alpha_2 = 0$, $\beta_2 = 0$, $\alpha_3 = 0$, $\beta_3 = 0$, $\gamma_3 = 0.1205$). In practice, we recommend starting with a low budget initial fit to gauge the need for increasing the number of iterations and reducing the fractional tolerance levels for the optimization procedure, and then adjusting these settings as needed to improve the fit.

In addition to estimating the self-correcting process, we need to train the conditional mark model for canopy volume (m^3) given the arrival times, locations, location specific topographic covariates, and interpoint competition metrics in a $10m$ neighborhood. We begin by loading the example raster images and scaling them.

```

raster_paths <- list.files(system.file("extdata", package = "ldmppr"),
                           pattern = "\\\\.tif$", full.names = TRUE)
raster_paths <- raster_paths[grepl("_med\\.tif$", raster_paths)]
rasters <- lapply(raster_paths, terra::rast)
scaled_rasters <- scale_rasters(rasters)

```

Next, to train the mark model, we opt for a gradient boosted tree model using the [xgboost](#) engine with 5-fold cross-validation and a hyperparameter tuning grid of size 50. We can pass the prepared data directly from the previous step, and specify the use of parallelization to speed up the training process. We use the selection metric of mean absolute error (MAE) to identify the optimal model from the tuning grid.

```

example_trained_mark_model <- train_mark_model(
  data = estimated_sc,
  raster_list = scaled_rasters,
  scaled_rasters = TRUE,
  model_type = "xgboost",
  parallel = TRUE,
  include_comp_inds = TRUE,
  competition_radius = 10,
  edge_correction = "none",
  selection_metric = "mae",

```

```

cv_folds = 5,
tuning_grid_size = 50,
verbose = FALSE
)

```

With the estimated parameters for the self-correcting process and a trained conditional mark model in hand, we now assess how well the estimated process reflects the dynamics in the original dataset using the `check_model_fit()` function. In order to obtain valid p -values at the $\alpha = .05$ level for the global envelope tests, [Myllymäki et al. \(2017\)](#) recommend using a minimum of 2499 realizations from the process, however for our initial fit we reduce the number of realizations to get an approximate sense of our performance. We pass the reference data and estimated process parameters from our estimation step through the `process_fit` argument, and the trained mark model and scaled rasters through the `mark_model` argument. We also include the competition indices in the simulations to match the training of the mark model.

```

example_model_fit <- check_model_fit(
  process = "self_correcting",
  process_fit = estimated_sc,
  mark_model = example_trained_mark_model,
  include_comp_inds = TRUE,
  thinning = TRUE,
  edge_correction = "none",
  competition_radius = 10,
  n_sim = 499,
  save_sims = FALSE,
  verbose = FALSE,
  parallel = FALSE,
  fg_correction = "km"
)

```

Once we have run the `check_model_fit()` function, we can plot the combined global envelope test, or any of the individual tests for the summary functions (L , F , G , J , E , or V), using the default `plot()` method as seen in Figure 1.

```
plot(example_model_fit)
```

In this example, our initial estimated model does not actually provide evidence of an adequate fit to the reference dataset as evidenced by the small p value ($p = 0.002$) and the number of points from the reference data that fall outside of the simulated envelopes (as highlighted in red). In particular, the nearest neighbor distance distribution function, $G(r)$, from the original data is not well captured by realizations from the fitted process. The $G(r)$ function is the distribution function of the distance from an arbitrary event to its nearest neighboring event, and is commonly used to assess the degree of clustering or inhibition in a point pattern relative to complete spatial randomness (CSR) ([Ripley, 1988](#)). We also note that the function $V(r)$, which captures the variance of the mark associated with a typical random point given that another random point exists at distance r , is not well captured by the fitted process. This suggests that the estimated mark model may not be capturing the dynamics of the mark process well, or that the self-correcting process is not adequately capturing the spatial dynamics of the reference dataset.

This result suggests that we may need to increase the budget and strategy for our self-correcting model estimation step when using the `estimate_process_parameters()` function, and potentially expand the training regime for our conditional mark model.

Given that we obtained a less than optimal fit with our initial modeling attempt, we revisit both the self-correcting process and mark models to see if we can improve them. A standard approach for improving estimation of the self-correcting process is to increase

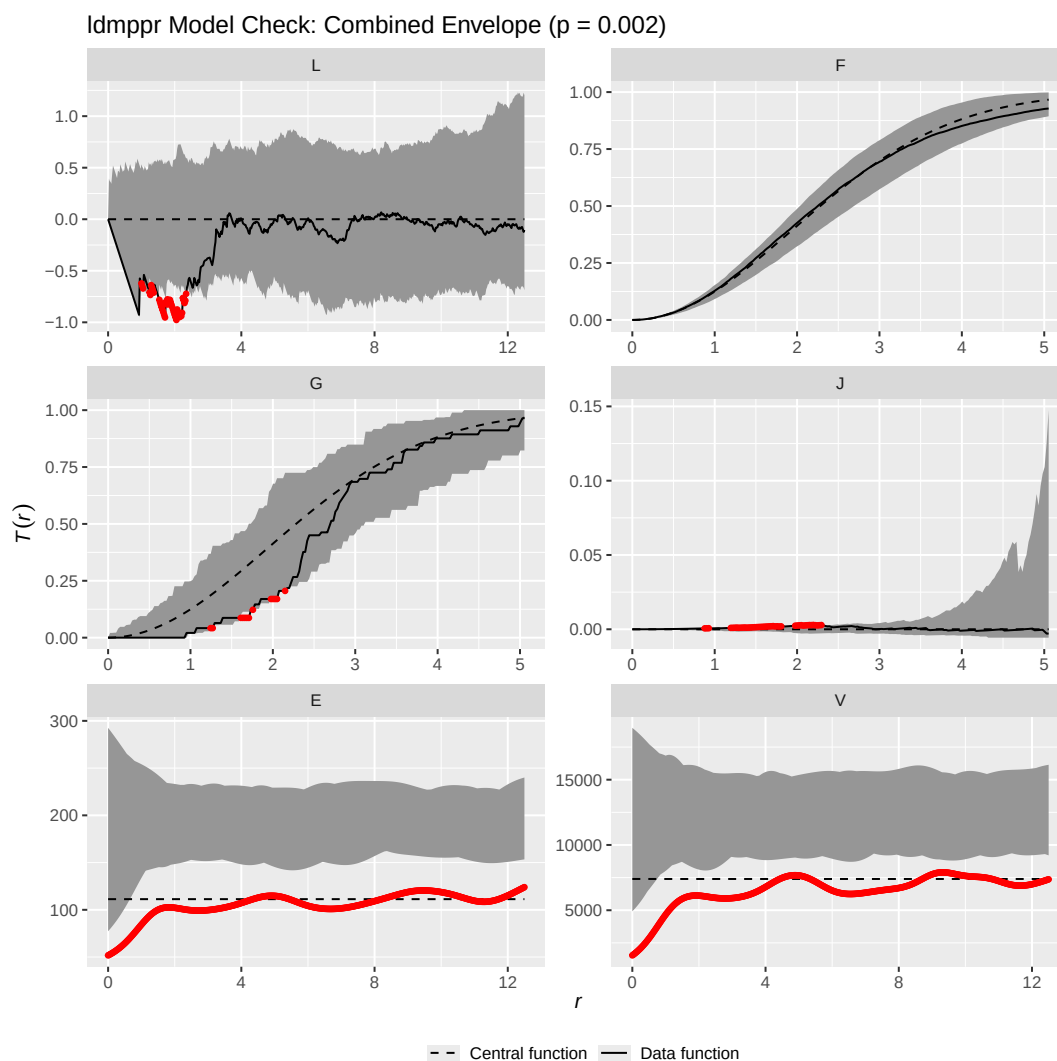


Figure 1: Combined global envelope test for the realizations from the fitted process. Solid black lines represent the reference process, dashed black lines represent a homogeneous Poisson process (or CSR), and the colored band represents the global envelope for the simulated datasets at the $\alpha = .05$ level. Reference values outside the envelope are highlighted in red and suggest a poor fit.

the granularity of the grid used to estimate the model, while increasing the number of iterations and potentially reducing the tolerance threshold. We can also leverage the “multires_global_local” strategy to improve the estimate iteratively, while making use of multiple starts at each level to expand our search area to identify a global optimum. We refit the model with a sequence of finer grid levels and increase the budget for the optimization procedure to allow for a more thorough search of the parameter space, and reduce the tolerance levels to facilitate convergence to a more polished solution. We also introduce additional starts at each optimization level to explore the parameter space more effectively and reduce our dependence on the initial values. We maintain the same optimization strategy and algorithms as before, and use the internally derived initial values and parameter bounds based on the reference data for the optimization.

```

grids <- ldmppr_grids(
  upper_bounds = upper_bounds,
  levels = list(c(10, 10, 10),
                c(12, 12, 12),
                c(16, 16, 16))
)

budgets <- ldmppr_budgets(
  global_options = list(
    maxeval = 1000,
    ftol_rel = 1e-6,
    xtol_rel = 1e-6
  ),
  local_budget_first_level = list(
    maxeval = 1500,
    ftol_rel = 1e-8,
    xtol_rel = 1e-8
  ),
  local_budget_refinement_levels = list(
    maxeval = 2000,
    ftol_rel = 1e-10,
    xtol_rel = 1e-10
  )
)

estimated_sc_update <- estimate_process_parameters(
  data = parameter_estimation_data,
  process = "self_correcting",
  grids = grids,
  budgets = budgets,
  delta = 1,
  parallel = FALSE,
  strategy = "multires_global_local",
  global_algorithm = "NLOPT_GN_CRSS2_LM",
  local_algorithm = "NLOPT_LN_BOBYQA",
  starts = list(global = 5, local = 3, jitter_sd = 0.15, seed = 90210),
  verbose = TRUE
)

improved_optimal_parameters <- coef(estimated_sc_update)
summary(estimated_sc_update)

```

We obtain the following improved parameter estimates from the optimization.

```

improved_optimal_parameters <- if (inherits(estimated_sc_update, "ldmppr_fit")) {
  coef(estimated_sc_update)
} else {
  as.numeric(estimated_sc_update$solution)
}
summary(estimated_sc_update)

#> Summary: ldmppr Fit
#>   process:      self_correcting
#>   engine:       nloptr
#>   strategy:     multires_global_local
#>   starts:       global=5, local=3, jitter_sd=0.15, seed=90210
#>   status:       3
#>   outcome:      NLOPT_FTOL_REACHED: Optimization stopped because ftol_rel or
#>                  ftol_abs (above) was reached.
#>   objective:    387.0972
#>   selected_delta: 1
#>   elapsed_sec:   1.935
#>   coefficients:
#> [1] 1.4405824 7.3756717 0.0314348 1.6980704 0.9817515 2.4976881 1.8469612
#> [8] 0.0832517

```

We see that the improved parameter estimates set ($\alpha_1 = 1.4406$, $\beta_1 = 7.3757$, $\gamma_1 = 0.0314$, $\alpha_2 = 1.6981$, $\beta_2 = 0.9818$, $\alpha_3 = 2.4977$, $\beta_3 = 1.847$, $\gamma_3 = 0.0833$) has shifted compared to the initial estimates.

Next, we retrain the mark model using two changes. First, we increase the size of the hyperparameter tuning grid to 150 to better explore the hyperparameter space and increase the number of cross-validation folds from 5 to 10, which typically reduces the variance of performance estimates to provide a more stable estimate of model generalization.

We then retrain the mark model using the updated estimated process and adjusted model specification.

```

improved_example_trained_mark_model <- train_mark_model(
  data = estimated_sc_update,
  xy_bounds = c(0, 50, 0, 50),
  raster_list = scaled_rasters,
  scaled_rasters = TRUE,
  model_type = "xgboost",
  parallel = TRUE,
  include_comp_inds = TRUE,
  competition_radius = 10,
  edge_correction = "none",
  selection_metric = "mae",
  cv_folds = 10,
  tuning_grid_size = 150,
  verbose = TRUE
)

```

We proceed by checking the fit of our model using the improved parameter estimates for the self-correcting process and the retrained mark model. We use the same reference dataset and raster images as before, and simulate a full 2500 realizations from the process to assess the fit of the updated model.

```

improved_example_model_fit <- check_model_fit(
  process = "self_correcting",
  process_fit = estimated_sc_update,

```

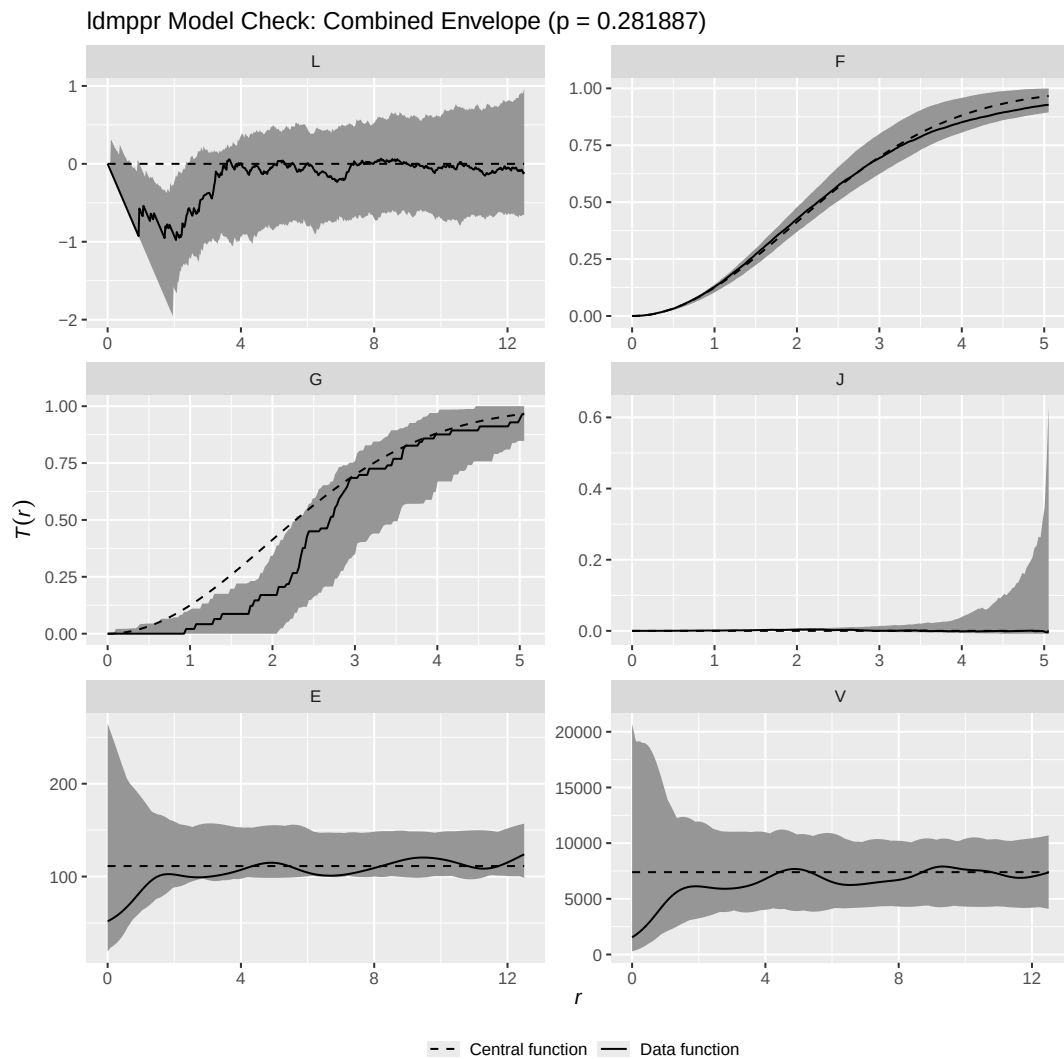


Figure 2: Combined global envelope test for the realizations from the improved fitted process. As in Figure 1, solid black lines represent the reference process, dashed black lines represent a homogeneous Poisson process (or CSR), and the colored band represents the global envelope for the simulated datasets at the $\alpha = .05$ level.

```
mark_model = improved_example_trained_mark_model,
include_comp_inds = TRUE,
thinning = TRUE,
edge_correction = "none",
competition_radius = 10,
n_sim = 2500,
save_sims = FALSE,
verbose = TRUE,
parallel = FALSE,
fg_correction = "km"
)
```

Next, we check the combined global envelope test for the updated model to assess how well the estimated model captures the dynamics of the reference dataset.

```
plot(improved_example_model_fit)
```

We see that the improved model provides a notably improved fit to the reference dataset, as seen in Figure 2, evidenced by the fact that the simulation envelopes contain the reference

pattern across all six metrics. In addition, the p -value for the combined global envelope test is $p = 0.2819$, indicating that the estimated process is a markedly improved fit to the reference dataset compared to the fit that we initially achieved.

When evaluating the fit of a model, in addition to the non-parametric summary statistics and global envelope tests, it may also be useful to perform a visual comparison of a realization from the model and the reference dataset. To assess the agreement between the improved fitted model and the reference data, we simulate a realization from the improved model and compare it to the original reference dataset and a dataset simulated from the initial (poorly fitting) model.

```
improved_simulated_mpp <- simulate_mpp(
  process = "self_correcting",
  process_fit = estimated_sc_update,
  mark_model = improved_example_trained_mark_model,
  include_comp_inds = TRUE,
  competition_radius = 10,
  edge_correction = "none",
  thinning = TRUE
)

initial_simulated_mpp <- simulate_mpp(
  process = "self_correcting",
  process_fit = estimated_sc,
  mark_model = example_trained_mark_model,
  include_comp_inds = TRUE,
  competition_radius = 10,
  edge_correction = "none",
  thinning = TRUE
)

ref_plot <- plot_mpp(
  mpp_data = reference_data,
  pattern_type = "reference"
)

improved_sim_plot <- plot(
  mpp_data = improved_simulated_mpp,
  pattern_type = "simulated"
)

initial_sim_plot <- plot(
  mpp_data = initial_simulated_mpp,
  pattern_type = "simulated"
)
```

Figure 3 demonstrates that the improved model provides a more accurate representation of the reference dataset than we obtained from the initial model in terms of the spatial process and the conditional mark process. We also include histograms of the realized mark distributions to highlight the improvement in replicating the mark distribution of the reference process. This visualization provides additional evidence that the improved model captures the spatial and mark dynamics of the reference dataset more effectively, which results in the improved model being a better fit. This example highlights the utility of the package for working with a real marked point process of interest, and provides the intuition for using the main functionality of **ldmppr** to estimate, assess model fit, and simulate from a marked spatial point process.

We note that the computational cost of the workflow is driven primarily by three steps:

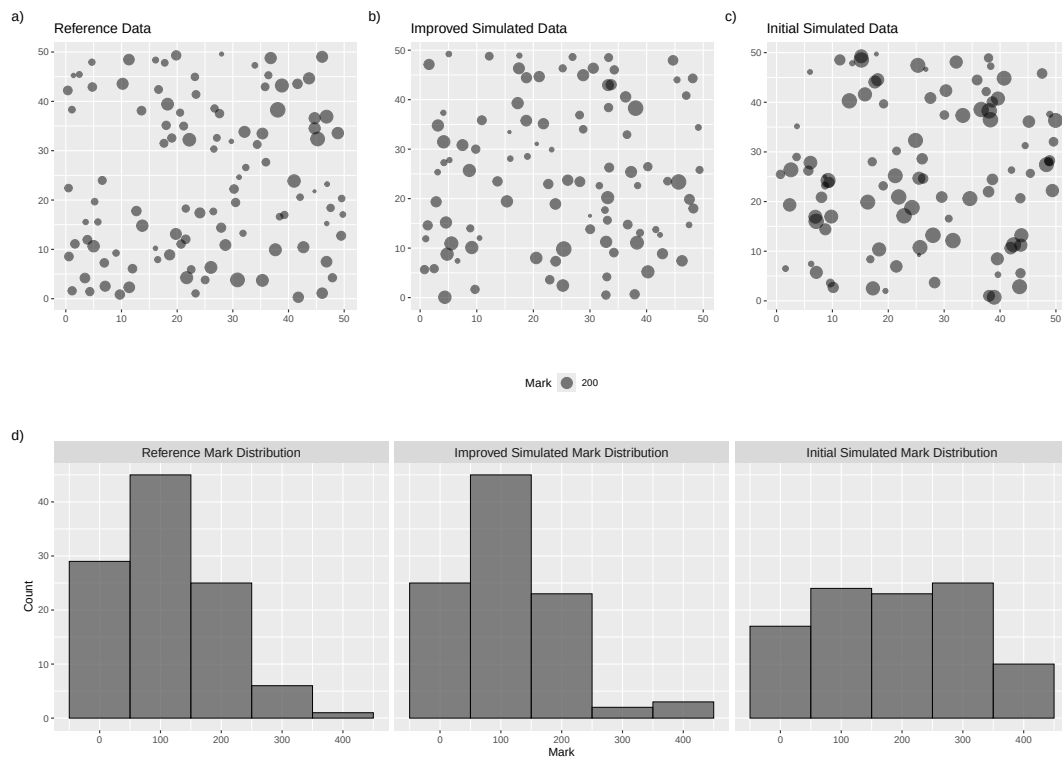


Figure 3: Plots a) - c) provide a comparison of a realization from the improved estimated process with the original reference dataset and a realization from the initial estimated process. Plot d) shows the corresponding observed mark distributions from the reference dataset and the simulation realizations.

self-correcting process estimation, mark-model training with cross-validation/tuning, and goodness-of-fit simulation in `check_model_fit()`. Runtime increases with grid resolution, optimizer budgets, number of starts, tuning grid size, cross-validation folds, and the number of simulations (`n_sim`). A practical approach is to begin with a low-cost baseline (coarser grid, fewer starts, smaller tuning grid, moderate `n_sim`) and increase complexity only when diagnostics indicate the need for refinement. Parallelization can also reduce wall-clock time for multi-start estimation, model tuning, and simulation-based checks when working with larger datasets, though the benefits are less pronounced for small datasets or lower budget runs. Concrete benchmark tables are provided in Appendices B and C of the Supplementary Materials for various usage scenarios, and Appendix D provides a contextual side-by-side comparison against a two-stage [spatstat](#) derived workflow under matched tuning/check settings.

5 Discussion

In this paper, we provide a novel framework for estimating location dependent marked point processes and introduce the [ldmppr](#) package, which contains a user-friendly modular suite of tools for model estimation, evaluation, simulation, and visualization for marked point processes with location dependence characterized by regularity in the spatial pattern. We outlined the typical workflow for using the package and discussed the key functions and their arguments in detail before providing an example of using the package with a real forestry dataset. [ldmppr](#) simplifies the process of working with marked point processes and provides a likelihood based estimation approach that is computationally feasible. While the framework presented applies to a broader range of marked point process, the package in its current implementation is still somewhat limited in the types of patterns that it can address. As the package continues to develop, we would like to incorporate additional models that can address point process data that demonstrates clustering behavior, as opposed to regularity, while still maintaining the focus on location dependent marks.

References

- A. Baddeley and R. Turner. spatstat: An R package for analyzing spatial point patterns. *Journal of Statistical Software*, 12(6):1–42, 2005. doi: 10.18637/jss.v012.i06. [p1]
- A. Baddeley, E. Rubak, and R. Turner. *Spatial Point Patterns: Methodology and Applications with R*. Chapman and Hall/CRC, New York, Nov. 2015. ISBN 978-0-429-16170-4. doi: 10.1201/b19708. [p12]
- F. L. Bayisa, M. Ådahl, P. Rydén, and O. Cronie. Regularised Semi-parametric Composite Likelihood Intensity Modelling of a Swedish Spatial Ambulance Call Point Pattern. *Journal of Agricultural, Biological and Environmental Statistics*, 28(4):664–683, Dec. 2023. ISSN 1537-2693. doi: 10.1007/s13253-023-00534-5. [p1]
- L. Breiman. Random Forests. *Machine Learning*, 45(1):5–32, Oct. 2001. ISSN 1573-0565. doi: 10.1023/A:1010933404324. [p5]
- T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, pages 785–794, New York, NY, USA, Aug. 2016. Association for Computing Machinery. ISBN 978-1-4503-4232-2. doi: 10.1145/2939672.2939785. [p5]
- T. Chen, T. He, M. Benesty, V. Khotilovich, Y. Tang, H. Cho, K. Chen, R. Mitchell, I. Cano, T. Zhou, M. Li, J. Xie, M. Lin, Y. Geng, Y. Li, J. Yuan, and D. Cortes. *xgboost: Extreme Gradient Boosting*, 2026. [p12]
- M. A. Contreras, D. Affleck, and W. Chung. Evaluating tree competition indices as predictors of basal area increment in western Montana forests. *Forest Ecology and Management*, 262(11):1939–1949, Dec. 2011. ISSN 0378-1127. doi: 10.1016/j.foreco.2011.08.031. [p12]
- D. J. Daley and D. Vere-Jones. *An Introduction to the Theory of Point Processes*. Springer, New York, NY, 1 edition, 1988. ISBN 978-0-387-96666-3. URL <https://link.springer.com/book/9780387966663>. [p3]
- P. J. Diggle. *Statistical Analysis of Spatial and Spatio-Temporal Point Patterns*. Chapman and Hall/CRC, New York, 3 edition, July 2013. ISBN 978-0-429-09809-3. doi: 10.1201/b15326. [p3, 12]
- P. J. Diggle and R. J. Gratton. Monte Carlo Methods of Inference for Implicit Statistical Models. *Journal of the Royal Statistical Society: Series B (Methodological)*, 46(2):193–212, Jan. 1984. ISSN 0035-9246. doi: 10.1111/j.2517-6161.1984.tb01290.x. [p5]
- L. Drew and A. Kaplan. *ldmppr: Estimate and Simulate from Location Dependent Marked Point Processes*, 2025. [p1]
- L. Drew, A. Kaplan, and I. Breckheimer. Data from "A Bayesian Record Linkage Approach to Applications in Tree Demography Using Overlapping LiDAR Scans", 2024. [p15]
- L. Drew, A. Kaplan, and I. Breckheimer. A Bayesian Record Linkage Approach to Applications in Tree Demography Using Overlapping LiDAR Scans. *The Annals of Applied Statistics*, 0(0):1–26, June 2025. URL <https://arxiv.org/search/?query=A+Bayesian+Record+Linkage+Approach+to+Applications+in+Tree+Demography+Using+Overlapping+LiDAR+Scans&searchtype=title>. [p15]
- D. S. Harte. PtProcess: An R package for modelling marked point processes indexed by time. *Journal of Statistical Software*, 35(8):1–32, 2010. doi: 10.18637/jss.v035.i08. [p1]
- V. Isham and M. Westcott. A self-correcting point process. *Stochastic Processes and their Applications*, 8(3):335–347, May 1979. ISSN 0304-4149. doi: 10.1016/0304-4149(79)90008-5. [p2, 4]

- S. G. Johnson. *The NLOpt Nonlinear-Optimization Package*, 2008. URL <https://nlopt.readthedocs.io/>. [p10]
- P. Kaelo and M. M. Ali. Some Variants of the Controlled Random Search Algorithm for Global Optimization. *Journal of Optimization Theory and Applications*, 130(2):253–264, Aug. 2006. ISSN 1573-2878. doi: 10.1007/s10957-006-9101-0. [p10]
- G. O. Mohler, M. B. Short, P. J. Brantingham, F. P. Schoenberg, and G. E. Tita. Self-Exciting Point Process Modeling of Crime. *Journal of the American Statistical Association*, 106(493):100–108, Mar. 2011. ISSN 0162-1459. doi: 10.1198/jasa.2011.ap09546. [p1]
- J. Møller and R. P. Waagepetersen. *Statistical Inference and Simulation for Spatial Point Processes*. Chapman and Hall/CRC, New York, Sept. 2003. ISBN 978-0-203-49693-0. doi: 10.1201/9780203496930. [p3, 14]
- J. Møller, M. Ghorbani, and E. Rubak. Mechanistic spatio-temporal point process models for marked point processes, with a view to forest stand data. *Biometrics*, 72(3):687–696, 2016. ISSN 1541-0420. doi: 10.1111/biom.12466. [p2, 3, 4, 5, 6]
- M. Myllymäki, T. Mrkvička, P. Grabarnik, H. Seijo, and U. Hahn. Global envelope tests for spatial processes. *Journal of the Royal Statistical Society. Series B (Statistical Methodology)*, 79(2):381–404, 2017. ISSN 1369-7412. doi: 10.1111/rssb.12172. [p12, 18]
- J. A. Nelder and R. Mead. A Simplex Method for Function Minimization. *The Computer Journal*, 7(4):308–313, Jan. 1965. ISSN 0010-4620, 1460-2067. doi: 10.1093/comjnl/7.4.308. [p10]
- A. Pommerening and A. J. Sánchez Meador. Tamm review: Tree interactions between myth and reality. *Forest Ecology and Management*, 424:164–176, Sept. 2018. ISSN 0378-1127. doi: 10.1016/j.foreco.2018.04.051. [p12]
- M. Powell. The BOBYQA algorithm for bound constrained optimization without derivatives. *Technical Report, Department of Applied Mathematics and Theoretical Physics*, Jan. 2009. URL <https://optimization-online.org/2010/05/2616/>. [p10]
- W. L. Price. Global optimization by controlled random search. *Journal of Optimization Theory and Applications*, 40(3):333–348, July 1983. ISSN 1573-2878. doi: 10.1007/BF00933504. [p10]
- S. L. Rathbun and N. Cressie. A space-time survival point process for a longleaf pine forest in southern georgia. *Journal of the American Statistical Association*, 89(428):1164–1174, 1994. ISSN 0162-1459. doi: 10.2307/2290979. [p3]
- B. D. Ripley. The second-order analysis of stationary point processes. *Journal of Applied Probability*, 13(2):255–266, June 1976. ISSN 0021-9002, 1475-6072. doi: 10.2307/3212829. [p12]
- B. D. Ripley. *Statistical Inference for Spatial Processes*. Cambridge University Press, Cambridge, 1988. ISBN 978-0-521-42420-2. doi: 10.1017/CBO9780511624131. [p18]
- T. H. Rowan. *Functional Stability Analysis of Numerical Algorithms*. PhD thesis, University of Texas at Austin, USA, 1990. URL <https://search.worldcat.org/search?q=Functional+Stability+Analysis+of+Numerical+Algorithms+Rowan>. [p10]
- M. Schlather, P. J. Ribeiro, and P. J. Diggle. Detecting Dependence between Marks and Locations of Marked Point Processes. *Journal of the Royal Statistical Society. Series B (Statistical Methodology)*, 66(1):79–93, 2004. ISSN 1369-7412. doi: 10.1046/j.1369-7412.2003.05343.x. [p2, 12, 14]
- M. N. M. van Lieshout. A J-Function for Marked Point Patterns. *Annals of the Institute of Statistical Mathematics*, 58(2):235–259, June 2006. ISSN 1572-9052. doi: 10.1007/s10463-005-0015-7. [p12]

M. N. Wright and A. Ziegler. ranger: A fast implementation of random forests for high dimensional data in C++ and R. *Journal of Statistical Software*, 77(1):1–17, 2017. doi: 10.18637/jss.v077.i01. [p12]

Lane Drew

Colorado State University

Department of Statistics, Colorado State University

851 Oval Dr, Fort Collins, CO 80524

<https://lanedrew.com/>

ORCID: 0009-0006-5427-4092

lane.drew@colostate.edu

Andee Kaplan

Colorado State University

Department of Statistics, Colorado State University

851 Oval Dr, Fort Collins, CO 80524

<https://andeekaplan.com/>

ORCID: 0000-0002-0912-0225

andee.kaplan@colostate.edu